install it on your system, then download the AppImage from *https://github.com/PowerShell/PowerShell/releases/tag/v6.0.0-alpha.17* (*PowerShell-x86_64.AppImage*). Follow the two steps given below. AppImage is a portable application, which contains the bundled PowerShell and dependencies:

```
chmod a+x PowerShell-x86_64.AppImage
./PowerShell-x86_64.AppImage
```

Alternatively, one can download the source code by running the following command:

```
git clone --recursive https://github.com/PowerShell/PowerShell.git
```

If you are interested in contributing to PowerShell, refer to the GitHub link *https://github.com/PowerShell/PowerShell/tree/master/docs/git*.

Let's run some cmdlets

Once the installation is successful, open a terminal and enter 'powershell'. This will open the PowerShell prompt in the same terminal. The first cmdlet we run will give us the PowerShell version information.

Let's try to find the number of cmdlets available with this alpha build of PowerShell. Currently, there are 349 cmdlets ported and more are on the way (refer to Figure 1).

Figure 2 gives another example where we pipe the result just like in Linux scripting.

Uninstalling PowerShell

Just run the following command to uninstall PowerShell:

```
Sudo apt-get remove powershell
```

Everything in PowerShell is cmdlets (verb-noun semantics)—for example, Get-Commands, Get-Process, Clear-Host, Set-Location, etc. Another good thing about PowerShell is that most of the Linux commands are already ported into it.

PowerShell vs Bash

The differences between the Bash shell and PowerShell are more with regard to the end-application use cases. Both are very strong and feature-rich automation languages but are inherently different in the way they perceive and handle the inputs and outputs. PowerShell deals with structured data in the form of objects, which can be pipelined and passed over to different commands, while the Bash script deals more with a bunch of strings that are not formatted or structured.

Linux native management based tasks are slowly being ported to PowerShell. Once this process is complete,

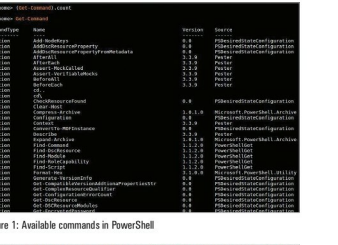

Figure 1: Available commands in PowerShell

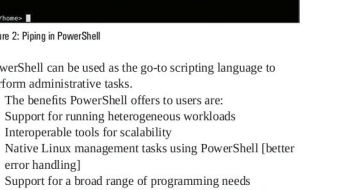

Figure 2: Piping in PowerShell

PowerShell can be used as the go-to scripting language to perform administrative tasks.

The benefits PowerShell offers to users are:

1. Support for running heterogeneous workloads
2. Interoperable tools for scalability
3. Native Linux management tasks using PowerShell [better error handling]
4. Support for a broad range of programming needs (interactive shell, scripting, system programming, etc) covered under a single umbrella
5. Optimised for structured data, as the workflow is object based

PowerShell is a new addition to the Linux environment. The use cases are not limited to the ones listed in this article. By open sourcing it, Microsoft has paved the way for Linux sub-systems to use MS based cloud products like Azure Portal, etc. Slowly, this will lead to standardisation between various OSs, as Bash and PowerShell will be supported across different operating systems. END

By: Vinay Patkar and Shubhra Rana

Vinay Patkar works as a software development engineer at Dell India R&D Centre, Bengaluru, and has close to two years' experience in automation and the Windows Server OS. He is interested in virtualisation and cloud computing technologies.

Shubhra Rana is a software development engineer at Dell India R&D Centre, Bengaluru, and is interested in data mining, network security and cryptography.